CSC 251 Lecture 06: Sequence Methods

Sequence Methods

General methods that apply to ALL Python sequences, both mutable and immutable, include:

- Indexing – using `[index]` notation
- Concatenation – using '+' and '+=" operators, a new sequence is returned
- Repetition – using '*' operator, a new sequence is returned
- Membership – using `in` and `not in`
- Slicing – using `[start:end:step]` notation, a new sequence is returned
- Length – using `len(sequence)`

Example 1a: when the index is needed to access the correct value

```
scores = [70,98,89,92,0,90,100,0,76]
index = 0
while index < len(scores):
   if scores[index] == 0:
     print(f"You missed homework #{index + 1}.")
   index += 1
```

- Iteration – using `for/in` notation

Example 1b:

```
holiday_str = 'Thanksgiving'
count = 0
for ch in holiday_str:
   if ch == 'T' or ch  == 't':
      count += 1
```

- Min/max – returns largest/smallest value from homogenous sequences

Example 1c:

```
nums = [90, 80, 100, 70, 60]
print(max(nums), min(nums))              100 60

print(max([5, 2, 3.14, 'hi']))           TypeError
```

no need to iterate

- Sorted – returns a **new** sorted list from the elements of any sequence

Example 1d:

```
num_tup = (1, 3, 2, 6, 4, 8, 0)
print(sorted(num_tup))                   [0, 1, 2, 3, 4, 6, 8]
```

What is output? What does this imply?

```
print(num_tup)
```

original (1, 3, 2, 6, 4, 8, 0)

Other Python methods are called on the sequence object using the notation:

sequence.method(arguments)

oop-style

member access operator

These include:

- Count – enumerates the number of occurrences of a value or substring without explicit iteration

Example 1e:

```
count_Ts = 'Titus'.count('T')          1
```

or

```
print(scores.count(0))                 2
```

- Index – returns the index of the first occurrence of a value or string

Example 1f:

```
print(holiday_str.index('i'))          7
print(holiday_str.index('in'))         9
```

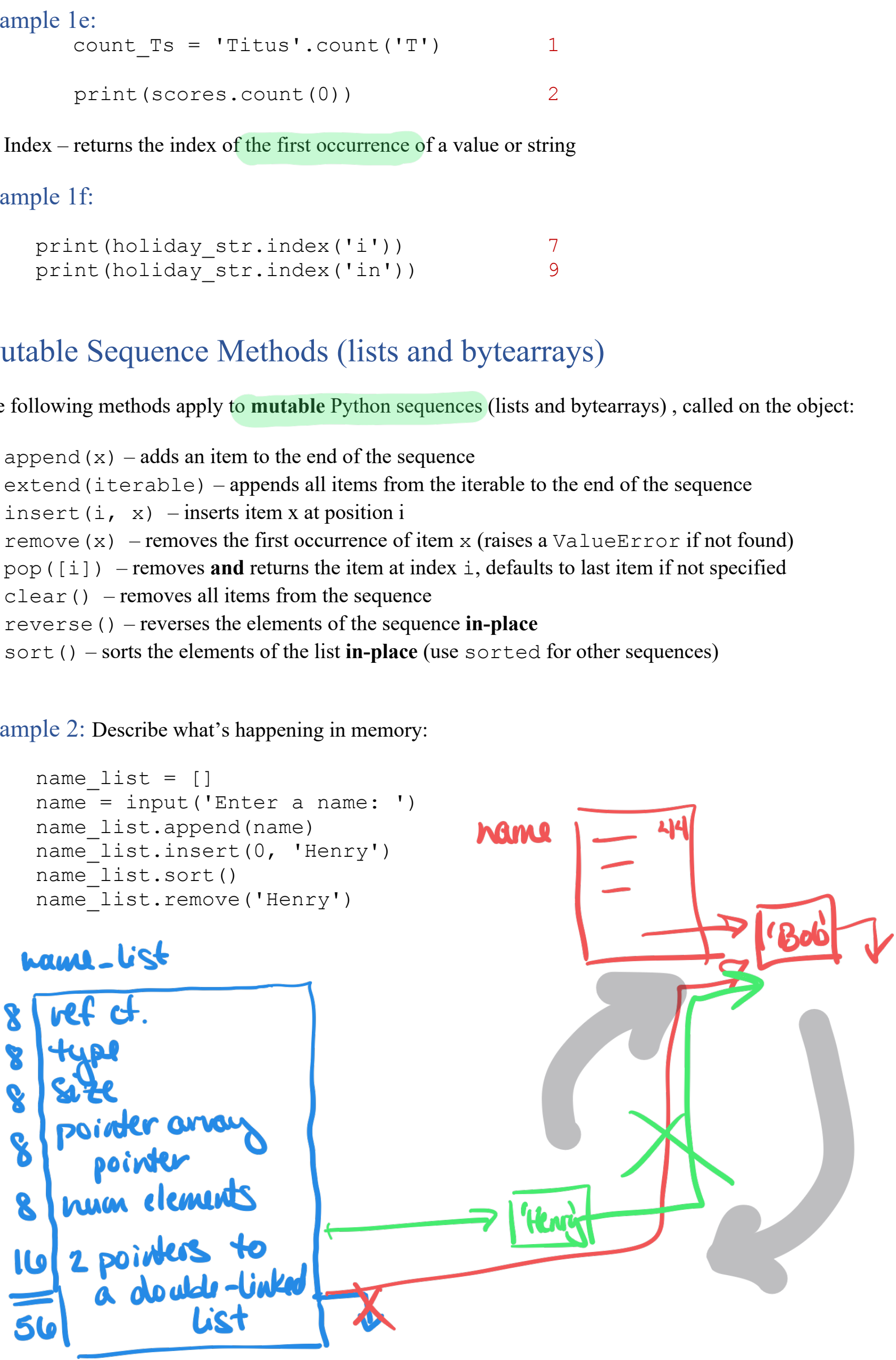
Mutable Sequence Methods (lists and bytearrays)

The following methods apply to **mutable** Python sequences (lists and bytearrays) , called on the object:

- `append(x)` – adds an item to the end of the sequence
- `extend(iterable)` – appends all items from the iterable to the end of the sequence
- `insert(i, x)` – inserts item x at position i
- `remove(x)` – removes the first occurrence of item `x` (raises a `ValueError` if not found)
- `pop([i])` – removes **and** returns the item at index `i`, defaults to last item if not specified
- `clear()` – removes all items from the sequence
- `reverse()` – reverses the elements of the sequence **in-place**
- `sort()` – sorts the elements of the list **in-place** (use `sorted` for other sequences)

Example 2: Describe what's happening in memory:

```
name_list = []
name = input('Enter a name: ')
name_list.append(name)
name_list.insert(0, 'Henry')
name_list.sort()
name_list.remove('Henry')
```



Passing a List to a Function

Example 3: Suppose a function computes the average of the list, passed in from main:

```
def get_average(num_list):
    total = 0
    for value in num_list:
        total += value
    return total/len(num_list)

def main():
    scores = [100, 78, 83, 91]
    print(f"Average = {get_average(scores):.1f}")

if __name__ == "__main__":
  main()
```

pass-by-(object)reference
modifications seen in main

Returning a List from a Function

Example 4: Create a list in the function then return it to the main function, storing it as a list there:

```
def main():
    numbers = get_values()
    print('The numbers in the list are:')
    print(numbers)

def get_values():
    values = []
    again = 'Y'
    while again.upper() == 'Y':
        num = int(input('Enter a number: '))
        values.append(num)
        print('Do you want to add another number?')
        again = input('y = yes, anything else = no: ')
    return values
```

must create a list so that you can append

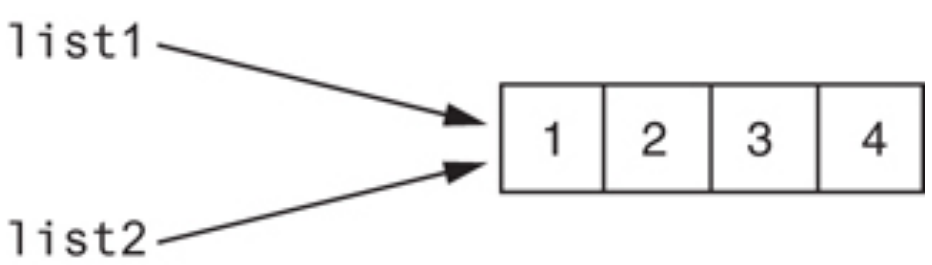
return to main (pass-by-ref)

Copying Lists

Example 5a: Consider the following list assignment:

```
list1 = [10, 20, 30, 40]
list2 = list1
```

Both `list1` and `list2` reference the SAME list in memory. If `list1` is changed, it changes `list2`.



shallow copy

Example 5b: If we prefer that `list1` and `list2` reference SEPARATE but IDENTICAL lists, we must make a copy of the list's elements:

deep copies

```
list1 = [10, 20, 30, 40]
list2 = []
for item in list1:
     list2.append(item)
```

Example 5c: OR, we can concatenate `list2` to `list1`:

preferred

```
list1 = [10, 20, 30, 40]
list2 = [] + list1
```

Example 5d: OR, initialize with `list1`:

```
list1 = [10, 20, 30, 40]
list2 = list(list1)
```

List Comprehensions

Python allows you to use a concise expression, called a list comprehension, to create a new list by iterating over an existing list:

```
list2 = [item for item in list1]
```

Result Expression — Iteration Expression

Example 6a: Suppose you wanted to create a list containing the doubles of another list:

```
listA = [10, 20, 30, 40]
listB = [item * 2 for item in listA]
```

Example 6b: Suppose the result expression was the length of each string in a list of strings:

```
names = ['Wicked', 'Cats', 'Phantom']
len_names = [len(x) for x in names]
```

Example 6c: You can also use an `if` clause with a list comprehension:

```
list1 = [5, 40, 90, 25, 10]

list2 = []
for item in list1:
    if item > 20:
        list2.append(item)
```

more efficiently:

```
list2 = [item for item in list1 if item < 20]
```

Two-dimensional Lists

Example 7: A list can contain nested lists:

```
student_list = [['Joe', 'Kim'], ['Sam', 'Sue'], ['Kelly', 'Chris']]
```

	Column 0	Column 1
Row 0	'Joe'	'Kim'
Row 1	'Jake'	'Sue'
Row 2	'Kelly'	'Chris'

Example 8a: Create the 2D list from input each iteration through a loop (assume an empty list already created):

```
first = input('Enter first name: ')
last = input('Enter last name: ')
student_list.append([last, first])
```

Example 8b: To change an element, specify *listName* [*row*] [*column*]

```
student_list[1][0] = 'Jake'
```

Example 8c: To grow the 2D list, append a list:

```
student_list.append(['Doug', 'Percy'])
```

Example 8d: The Pythonic way to print out the 2D list:

```
for row in student_list:
    for element in row:
        print(element)
```

row is a sublist
element is an element of the sublist

Python Methods for Tuples

Tuples support the methods applicable to all sequences (listed above) but do NOT support any method that changes the tuple such as:

- `append`
- `remove`
- `insert`
- `reverse`
- `sort` (although `sorted` is works since it returns a sorted list of the tuple items)

Why use tuples?

1) Uses memory more efficiently since less padding
2) Speed - faster to create and iterate over since stored in contiguous memory
3) A precaution against changing elements
4) Convenience of packing/unpacking elements

Tuple Comprehensions

While there is no "tuple comprehension" in Python, you can either use list comprehension then convert to a tuple, or use a generator function to emulate tuple comprehension:

Example 9: To create a tuple of the cubes from 0 through 20:

```
tupA = tuple(val**3 for val in range(21))
```

work around with inline loop and constructor

String Methods

A complete list of methods can be found in the Python documentation or at:
https://www.w3schools.com/python/python_ref_string.asp

String methods fall into 3 categories:

1. String "modification" – returns a new string with modifications that should be assigned to a variable or used in an output statement. A few of these include:

   - `lower()` – converts all chars to lowercase
   - `upper()` – converts all chars to uppercase
   - `capitalize()` – converts the first char to uppercase and the rest to lowercase
   - `title()` – converts the first char of each word to lowercase
   - `strip()` – removes leading or trailing whitespace (or other specified chars) from the string, also `lstrip()` and `rstrip()` for stripped just from the left or right
   - `split()` – splits the list into a list of substrings using whitespace (default) or a specified separator

Example 10a: Using whitespace as the separator

```
message = "Welcome to Raleigh!"
mess_list = message.split()
print(mess_list)                    ["Welcome", "to", "Raleigh!"]
```

Example 10b: Using a specified separator:

```
date_string = "09/08/2025"
date_list = date_string.split('/')
print(date_list)                    ["09", "08", "2025]
```

- `join(iterable)` – concatenates the elements of an iterable, such as a list of strings, into a single string, using the string called upon as the separator.

Example 10c: Using a specified separator:

```
names = ["John", "Jacob", "Jingleheimer", "Schmidt"]
print("*".join(names))                    John*Jacob*Jingleheimer*Schmidt
```

2. String searching, besides the index(substring) and count(substring) methods:

- `find(substring)` – searches for the first occurrence of the substring and returns its index, much like the `index` method, but returns a -1 (rather than an error) if not found
- `startswith(prefix)` – returns True if the string starts with specified substring, False if not
- `endswith(suffix)` – returns True if the string ends with specified substring, False if not

Example 10d: Don't forget about case sensitivity!

```
print(holiday_str.startswith("thanks")) False
```

3. String Boolean checks – return `True` if all characters in the string satisfy the condition, `False` otherwise. A few of these methods:

- `isalnum()` – checks that all chars are alphanumeric
- `isalpha()` – checks if all chars are alphabetic
- `isdigit()` – checks if all chars are numeric digits
- `isspace()` – checks if all chars are whitespace chars
- `islower()` – checks if all chars are lowercase
- `isupper()` – checks if all chars are uppercase

Example 10e:

```
id = input('Enter id number: ')
if not id.isdigit():
    print(f'Invalid id number entered.')
```

Example 10f:

```
lowercase = 'asap'
print(lowercase.upper())
```

Example 11: Try chaining methods for maximum efficiency:

```
def count_T3(my_string):
    new_str = my_string.upper()
    new_new_str = new_str.count('T')
    return new_new_string
```

Versus:

```
def count_T3(my_string):
    return my_string.upper().count('T')
```

chained

Example 12:

```
def valid_password(password):
  # Set  Boolean flags to false
  correct_length = False
  has_uppercase = False
  has_lowercase = False
  has_digit = False

  # Test the password's length
  if len(password) >= 7:
      correct_length = True

      # Test each character and adjust flag value
      for ch in password:
          if ch.isupper():
              has_uppercase = True
          if ch.islower():
              has_lowercase = True
          if ch.isdigit():
              has_digit = True

  # Return true if all requirements are met; false if not
  return correct_length and has_uppercase and has_lowercase and has_digit
```

What's a good call statement?

print("valid:", valid_password("abc123"))